

Backend Design Document

Bootcamp Assignment & Submission Platform

Go Gin + MySQL | Tanpa Docker & Redis untuk Versi Awal

Nama Kandidat	Anggi Juhnawan
Role Assessment	Trainer Candidate Technical Assessment
Topik	Go Backend System Design Test
Tujuan Dokumen	Menjelaskan desain backend secara singkat, praktis, dan sederhana.

1. Problem Summary

Sistem ini adalah platform untuk mengelola tugas bootcamp, mulai dari pembuatan class/batch, assignment, submission talent, review trainer, feedback, revisi, sampai progress tracking.

- Admin mengelola class/batch, trainer, dan talent.
- Trainer membuat assignment, memeriksa submission, memberi nilai, feedback, dan meminta revisi.
- Talent melihat assignment sesuai class-nya dan mengumpulkan tugas melalui GitHub URL, deployment URL, atau link lain yang valid.

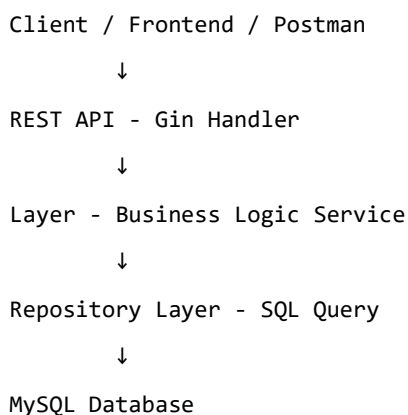
Tujuan sistem adalah membuat proses assignment dan review menjadi lebih terstruktur, mudah dimonitor, dan data progress talent dapat dilihat oleh trainer/admin.

2. Assumptions

- Satu user memiliki satu role utama: admin, trainer, atau talent.
- Satu class/batch dapat memiliki banyak trainer dan banyak talent.
- Satu assignment dibuat untuk satu class.
- Talent hanya dapat melihat assignment dari class tempat dia terdaftar.
- Talent dapat submit ulang hanya jika status submission adalah `revision_required`.
- Trainer hanya dapat review submission dari class yang dia handle.
- Admin memiliki akses penuh untuk mengelola class, trainer, dan talent.
- Database menggunakan MySQL 8.
- Untuk versi awal tidak menggunakan Redis/cache supaya data submission dan review tetap akurat.
- Untuk versi awal tidak menggunakan Docker.

3. Architecture Overview

Arsitektur backend menggunakan pendekatan monolith sederhana dengan pemisahan layer handler, service, repository, dan database.



Layer	Tanggung Jawab
Handler	Menerima HTTP request, membaca parameter/body, memanggil service, dan mengirim response.
Service	Berisi business logic seperti validasi role, aturan submit, aturan review, dan status transition.
Repository	Berisi query SQL ke MySQL, termasuk SELECT, INSERT, UPDATE, dan transaction query.
Database	Menyimpan data users, classes, assignments, submissions, reviews, dan histories.

Alasan memilih arsitektur ini adalah sederhana, mudah dites, dan mudah dijelaskan. Business logic tidak dicampur langsung di handler agar kode lebih maintainable.

4. API Endpoint List

Area	Method & Endpoint	Keterangan
Authentication	POST /api/v1/auth/login	Login user dan menghasilkan JWT token.
Authentication	GET /api/v1/auth/me	Mengambil profil user yang sedang login.
Class	POST /api/v1/classes	Admin membuat class/batch.
Class	GET /api/v1/classes	Melihat daftar class dengan pagination.
Class	GET /api/v1/classes/:id	Melihat detail class.
Class	POST /api/v1/classes/:id/trainers	Admin assign trainer ke class.
Class	POST /api/v1/classes/:id/talents	Admin assign talent ke class.
Assignment	POST /api/v1/classes/:classId/assignments	Trainer membuat assignment untuk class.
Assignment	GET /api/v1/classes/:classId/assignments	List assignment pada class.
Assignment	GET /api/v1/assignments/:id	Detail assignment.
Assignment	PUT /api/v1/assignments/:id	Trainer update assignment.
Assignment	PATCH /api/v1/assignments/:id/close	Trainer menutup assignment.
Submission	POST /api/v1/assignments/:assignmentId/submissions	Talent submit assignment.
Submission	PUT /api/v1/submissions/:id	Talent resubmit jika revision_required.
Submission	GET /api/v1/submissions/me	Talent melihat submission miliknya.
Submission	GET /api/v1/classes/:classId/submissions	Trainer melihat submission class.
Review	POST /api/v1/submissions/:id/review	Trainer memberi score dan feedback.
Review	POST /api/v1/submissions/:id/request-revision	Trainer meminta revisi.
Progress	GET /api/v1/classes/:classId/progress	Progress summary per class.
Progress	GET /api/v1/talents/:talentId/progress	Progress summary per talent.

Pagination Approach

Pagination digunakan untuk endpoint yang mengembalikan banyak data, seperti list classes, assignments, submissions, dan progress summary. Pendekatan yang dipakai adalah page-limit karena mudah dipahami, mudah dites melalui Postman, dan cukup untuk versi awal sistem.

Contoh query pagination:

GET /api/v1/classes?page=1&limit=10

GET /api/v1/classes/:classId/submissions?page=1&limit=10&status=submitted

Parameter	Keterangan
page	Nomor halaman yang diminta. Default: 1.
limit	Jumlah data per halaman. Default: 10 dan maksimal 100 agar query tidak terlalu berat.
sort_by	Kolom sorting yang diizinkan melalui whitelist, misalnya created_at atau deadline.
sort_order	Arah sorting: asc atau desc. Default: desc.

Contoh response metadata:

```
{
  "data": [],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 100,
    "total_pages": 10
  }
}
```

5. Database Schema or ERD

Database menggunakan MySQL. Relasi utama dibuat menggunakan foreign key agar data lebih konsisten.

Table	Kolom Utama	Keterangan
users	id, name, email, password_hash, role, created_at, updated_at	Menyimpan data admin, trainer, dan talent.
classes	id, name, description, start_date, end_date, created_at, updated_at	Menyimpan data class/batch.
class_trainers	id, class_id, trainer_id, created_at	Pivot table relasi class dengan trainer.
class_talents	id, class_id, talent_id, created_at	Pivot table relasi class dengan talent.
assignments	id, class_id, title, description, deadline, status, created_by, created_at, updated_at	Menyimpan tugas yang dibuat trainer.
submissions	id, assignment_id, talent_id, github_url, deployment_url, notes, status, submitted_at, reviewed_at, created_at, updated_at	Menyimpan hasil pengumpulan tugas talent.
submission_reviews	id, submission_id, trainer_id, score, feedback, review_status, created_at, updated_at	Menyimpan nilai dan feedback dari trainer.
submission_histories	id, submission_id, old_status, new_status, note, created_by, created_at	Menyimpan riwayat perubahan status submission.

Status penting:

Entity	Status
users.role	admin, trainer, talent
assignments.status	active, closed, archived
submissions.status	not_submitted, submitted, reviewed, revision_required, late

Relasi utama:

classes 1 - N assignments
classes N - N trainers melalui class_trainers
classes N - N talents melalui class_talents
assignments 1 - N submissions
users/talent 1 - N submissions
submissions 1 - N submission_reviews
submissions 1 - N submission_histories

Indexes & Constraints

Constraint dan index berikut disarankan agar data konsisten dan query utama tetap efisien.

Jenis	Detail	Alasan
UNIQUE	users.email	Mencegah email/account duplikat.
UNIQUE	class_trainers(class_id, trainer_id)	Trainer tidak bisa diassign dua kali ke class yang sama.
UNIQUE	class_talents(class_id, talent_id)	Talent tidak bisa diassign dua kali ke class yang sama.
UNIQUE	submissions(assignment_id, talent_id)	Satu talent hanya memiliki satu submission utama per assignment; resubmit dilakukan dengan UPDATE dan dicatat di history.
INDEX	assignments.class_id	Mempercepat list assignment berdasarkan class.
INDEX	submissions.assignment_id, submissions.talent_id, submissions.status	Mempercepat pencarian submission, review list, dan progress tracking.
INDEX	assignments.deadline	Membantu proses pengecekan deadline, reminder, atau mark late.
FOREIGN KEY	Semua kolom *_id mengarah ke tabel parent yang sesuai	Menjaga konsistensi relasi antar data.

Catatan: unique constraint pada submissions(assignment_id, talent_id) membuat proses resubmit lebih aman karena talent tidak membuat baris submission baru berkali-kali. Perubahan link/status dapat dicatat di submission_histories.

6. Main Request Flows

Flow Talent Submit Assignment

1. Talent login dan mendapatkan JWT token.
2. Talent membuka assignment dari class tempat dia terdaftar.
3. Talent mengirim GitHub URL/deployment URL.
4. Sistem validasi membership talent pada class tersebut.
5. Sistem cek deadline assignment.
6. Sistem menyimpan submission.
7. Status menjadi submitted jika tepat waktu, atau late jika melewati deadline.

Flow Trainer Review Submission

1. Trainer login dan membuka daftar submission class yang dia handle.
2. Trainer memilih submission talent.
3. Trainer memberi score dan feedback.
4. Sistem validasi trainer memang terdaftar pada class tersebut.
5. Sistem melakukan transaction: insert review, update status submission, insert history.
6. Status submission menjadi reviewed.

Flow Request Revision

1. Trainer membuka submission.
2. Trainer memberi feedback revisi.
3. Sistem mengubah status menjadi revision_required.
4. Talent dapat submit ulang dengan link yang sudah diperbaiki.

7. Go Project Structure

```
cmd/  
  api/  
    main.go  
internal/  
  config/  
    config.go  
  database/  
    mysql.go  
  middleware/  
    auth_middleware.go  
    role_middleware.go  
  handler/  
    auth_handler.go  
    class_handler.go  
    assignment_handler.go
```

```
    submission_handler.go
    review_handler.go
service/
    auth_service.go
    class_service.go
    assignment_service.go
    submission_service.go
    review_service.go
repository/
    user_repository.go
    class_repository.go
    assignment_repository.go
    submission_repository.go
    review_repository.go
model/
    user.go
    class.go
    assignment.go
    submission.go
dto/
    auth_dto.go
    assignment_dto.go
    submission_dto.go
utils/
    jwt.go
    response.go
    password.go

db/
    migrations/
        000001_init_schema.up.sql
        000001_init_schema.down.sql

README.md
.env.example
go.mod
```

Folder/File	Fungsi
cmd/api/main.go	Entry point aplikasi Gin.
internal/handler	Menerima request dan mengirim response.
internal/service	Business logic utama.
internal/repository	Query database.
internal/middleware	JWT auth dan role checking.
internal/dto	Struktur request/response.
db/migrations	SQL migration up dan down.

8. Authentication and Authorization Design

Authentication menggunakan JWT. Setelah login berhasil, client menyimpan token dan mengirimkannya pada setiap request yang membutuhkan login.

Authorization: Bearer <token>

Role	Hak Akses
Admin	Create/list/update class, assign trainer, assign talent.
Trainer	Create/update assignment, view submission class yang dia handle, review, request revision.
Talent	View assignment class-nya, submit assignment, resubmit jika revision_required, view submission miliknya.

Validasi authorization penting:

- Talent tidak boleh melihat submission milik talent lain.
- Talent tidak boleh submit assignment dari class lain.
- Trainer tidak boleh review submission dari class yang bukan dia handle.
- Admin boleh mengelola data class, trainer, dan talent.

9. Validation and Error Handling Strategy

Jenis Validasi	Contoh
Request validation	email wajib diisi, password wajib diisi, title wajib diisi, deadline wajib diisi, score 0-100.
Business validation	talent harus terdaftar di class, trainer harus handle class, assignment tidak boleh closed, status harus valid.

Format error response dibuat konsisten:

```
{
  "success": false,
  "message": "Unauthorized access",
}
```



```
"error": "trainer is not assigned to this class"
}
```

HTTP Status	Kondisi
400 Bad Request	Input tidak valid.
401 Unauthorized	Belum login atau token salah.
403 Forbidden	Role tidak punya akses.
404 Not Found	Data tidak ditemukan.
409 Conflict	Konflik status/data, misalnya submission sudah reviewed.
500 Internal Server Error	Error tidak terduga di server.

10. Transaction Handling Strategy

Transaction digunakan ketika satu proses mengubah lebih dari satu table. Tujuannya agar data tidak setengah berhasil.

Contoh proses yang membutuhkan transaction:

- Trainer review submission.
- Trainer request revision.
- Talent resubmit assignment.
- Admin assign trainer/talent ke class.

Contoh transaction saat trainer review:

```
BEGIN TRANSACTION
```

1. Lock data submission menggunakan `SELECT ... FOR UPDATE`.
2. Validasi status masih submitted atau late.
3. Insert data ke `submission_reviews`.
4. Update `submissions.status` menjadi `reviewed`.
5. Insert data ke `submission_histories`.

```
COMMIT
```

Jika salah satu proses gagal, jalankan `ROLLBACK`.

11. Caching Decision

Untuk versi awal, sistem tidak menggunakan Redis atau caching.

Alasan:

- Data submission, review, score, dan feedback harus akurat.
- Cache dapat menyebabkan stale data jika tidak di-invalidate dengan benar.
- MySQL sudah cukup untuk kebutuhan awal assessment.
- Redis adalah optional dan dapat ditambahkan nanti jika traffic meningkat.

Endpoint yang mungkin cocok dicache di masa depan:

- GET /api/v1/classes
- GET /api/v1/classes/:classId/assignments
- GET /api/v1/classes/:classId/progress
- GET /api/v1/classes/:id/submissions
- POST /api/v1/auth/login
- GET /api/v1/auth/me
- GET /api/v1/assignments/:id

Data yang tidak dicache di awal: submission status, score, feedback, revision status, dan review result.

12. Concurrency and Race Condition Consideration

Potensi Masalah	Solusi
Talent klik submit dua kali dengan cepat.	Gunakan unique constraint assignment_id + talent_id, transaction, dan cek status sebelum insert/update.
Dua trainer review submission yang sama.	Gunakan SELECT ... FOR UPDATE dan validasi status terbaru sebelum update.
Talent resubmit saat trainer sedang review.	Batasi resubmit hanya saat status revision_required dan review hanya saat submitted/late.
Background job mark late bersamaan dengan submit.	Gunakan transaction dan cek deadline/status terbaru.

13. Background Job Consideration

Background job tidak wajib untuk versi paling sederhana, tetapi ada beberapa proses yang cocok dikerjakan secara terjadwal.

Background Job	Keterangan
Mark late submissions	Menandai assignment/submission yang melewati deadline.
Reminder before deadline	Mengirim reminder kepada talent sebelum deadline.
Recalculate progress summary	Menghitung ulang progress jika nanti progress disimpan sebagai

	summary.
Clean expired tokens	Membersihkan refresh token jika sistem memakai refresh token.

Untuk versi awal, background job paling penting adalah mark late submissions. Namun status late juga bisa dihitung saat talent submit atau saat progress dibuka agar sistem tetap sederhana.

14. Testing Plan

Jenis Test	Yang Dites
Unit test	Service logic, role validation, status transition, validation score, dan business rule.
Integration test	Login, create class, assign trainer/talent, create assignment, submit, review, request revision, resubmit.
API test	Endpoint dites menggunakan Postman atau curl.
Authorization test	Talent tidak bisa akses data orang lain; trainer tidak bisa review class lain; admin bisa manage class.
Transaction test	Jika insert review gagal, status submission rollback; jika update status gagal, review tidak tersimpan.

15. Deployment Consideration

Untuk versi awal, deployment dibuat sederhana tanpa Docker.

1. Install MySQL di server.
2. Buat database bootcamp_assignment.
3. Set environment variable (.env atau systemd Environment).
4. Jalankan SQL migration: migrate up.
5. Build aplikasi Go: go build -o assignment-api ./cmd/api.
6. Jalankan aplikasi menggunakan systemd.
7. Gunakan Nginx sebagai reverse proxy jika dibutuhkan.

Environment yang dibutuhkan:

```
APP_PORT=8080
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=bootcamp_assignment
JWT_SECRET=your_secret_key
```

Untuk production, perlu memperhatikan HTTPS, backup database, logging, monitoring, dan secret management.

16. Trade-offs and Limitations

Trade-offs:

- Menggunakan monolith sederhana, bukan microservices, karena requirement masih kecil dan lebih mudah dijelaskan.
- Menggunakan MySQL tanpa Docker untuk versi awal agar setup lebih mudah.
- Tidak menggunakan Redis di awal karena menjaga data submission/review tetap akurat.
- SQL migration dipilih dibanding AutoMigrate agar perubahan database versioned, bisa direview, dan bisa rollback.

Limitations versi awal:

- Belum ada frontend.
- Belum ada file upload langsung; submission masih berupa link.
- Belum ada email notification.
- Belum ada queue system.
- Belum ada Redis cache.
- Belum ada analytics dashboard yang kompleks.

Improvement ke depan: Redis untuk progress summary, background worker untuk reminder, file storage, refresh token, Docker, dan monitoring.

17. AI / Tools Used and What I Personally Verified

AI/tools digunakan sebagai alat bantu untuk:

- Menyusun struktur design document berdasarkan 17 expected deliverables.
- Merapikan daftar endpoint REST API.
- Membantu membuat contoh database schema dan relasi utama.
- Membantu menyusun penjelasan transaction, caching, concurrency, dan background job.

Hal yang saya verifikasi sendiri:

- Role Admin, Trainer, dan Talent sesuai requirement.
- Endpoint mencakup authentication, class, assignment, submission, review, dan progress tracking.
- Database schema memiliki relasi utama: class, assignment, talent, submission, review, dan history.
- Submission status mendukung not_submitted, submitted, reviewed, revision_required, dan late.
- MySQL mendukung foreign key, transaction, index, dan row-level locking.
- Redis tidak wajib untuk versi awal dan dapat ditambahkan jika kebutuhan read-heavy meningkat.